

Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software.

Pros: The sheer joy of making things. The pleasure of making things that are useful to other people. The fascination of complex puzzle-like objects of interlocking moving parts and watching them work in subtle cycles. The joy of always learning. The delight of working in such a tractable medium.

Cons: One must perform perfectly. Other people set one’s objectives, provide one’s resources, and furnish one’s information. Designing grand concepts is fun; Finding nitty little bugs is just work. Debugging has a linear convergence, or worse. The product over which one has labored so long appears to be obsolete upon (or before) completion. Coding is still a ‘design’ activity, not a ‘manufacturing’ activity.

Object Oriented Programming (OOP)

OOP views the world as a network of interacting objects and is a programming paradigm.

Feature	Procedural	Functional	Logic
Focus	Step-by-step	Math Function	Logic Reasoning
State	Mutable	Immutable	No explicit state
Control	Loops, if/else	Recursion, etc	Control by NLP

OOP guides us in how to structure the solution. It is mainly an abstraction mechanism and NOT a programming language. It is modeled after how the objects in real world work. OOP is a higher-level mechanism (Objects) than the procedural paradigm (Data Structures). An **object** in OOP has **state** (data) and **behaviour** (operations on data). Every real-world object has an **interface** through which other objects can interact with it, and an **implementation** (internal code) that supports the interface but may not be accessible to other object.

The concept of Objects in OOP is an **abstraction mechanism** because it allows us to abstract away the lower-level details and work with bigger granularity entities. An object is an **encapsulation** of some data and related behaviour in terms of two aspects:

- 1. The packaging aspect: An object packages data and related behaviour together into one self-contained unit.
- 2. The information hiding aspect: The data in an object is hidden from the outside world and are only accessible using the object's interface. A **class** contains instructions for creating a specific kind of objects. Variables whose value is shared by all instances of a class are called **class-level/static attributes**. Methods that are called using the class instead of a specific instance are called **class-level/static methods**.

An **Enumeration** is a fixed set of values that can be considered as a data type. **Associations** are connections between objects. They can change over time and among objects, they can be generalized as associations between the corresponding classes too. **Navigability** is the concept of which object in the association knows about (has a reference to) the other object. It can be unidirectional or bidirectional. **Multiplicity** is the aspect of an OOP solution that dictates how many objects take part in each association. A **dependency** is a need for one class to depend on another without having a direct association in the same direction. A **composition** is an association that represents a strong whole-part relationship. When the whole is destroyed, parts are destroyed too. There cannot be cyclical links. **Aggregation** represents a container-contained relationship. It is a weaker relationship than composition. **Association class** represents additional information about an association. It is a normal class but plays a special role from a design point of view. **Inheritance** represents a “is a” relationship between two entities and allows you to define a new class based on an existing class.

A superclass is said to be more general than the subclass. Inheritance relationships through a chain of classes can result in inheritance hierarchies (aka inheritance trees). **Multiple Inheritance** is when a class inherits directly from multiple classes (Python, C++).

Method overloading is when there are multiple methods with the same name but different type signatures. An **interface** is a behaviour specification i.e. a collection of method specifications. If a class implements it (also an “is a” relationship, just like in class inheritance), it should be able to support the behaviours specified by the said interface. An **abstract class** cannot be instantiated, but it can be subclassed. Declared when a class is merely a representation of commonalities among its subclasses (No need to instantiate the class). An **abstract method** is a method signature without a method implementation and any class that has it becomes an abstract class. **Substitutability**: Every instance of a subclass is an instance of the superclass, but not vice-versa. **Dynamic binding** (aka **late binding**): A mechanism where method calls in code are resolved at runtime, rather than at compile time. **Overridden methods** are resolved using dynamic binding, and therefore resolves to the implementation in the actual type of the object. **Static binding** (aka **early binding**): When a method call is resolved at compile time. **Overloaded methods** are resolved using static binding. **Polymorphism** is the ability of different objects to respond, each in its own way, to the same method call. To achieve polymorphism, we need Substitutability, Operation Overriding (NOT Overloading), and Dynamic Binding.

Requirements

A **software requirement** specifies a need to be fulfilled by the software product. A software project may be a **brownfield** project (Develop a product to replace/update an existing software product) or a **greenfield** project (Develop a totally new system from scratch). In either case, requirements **from stakeholders** need to be gathered, analysed, specified, and managed. **Functional requirements** specify what the system should do. **Non-functional requirements** (NFR) specify the constraints under which the system is developed and operated. For example, the system should work on both 32-bit and 64-bit environments, the system should be usable by a novice who has never carried out an online purchase. There may be a **need** to spend extra effort to dig out NFRs ASAP as they are easy to miss and sometimes, critical to the success of the software. Requirements **can be prioritized** based on the **importance** and **urgency**, while keeping in mind the constraints of schedule, budget, staff resources, quality goals, and other constraints. **Characteristics of well-defined requirements** includes Unambiguous, Testable (verifiable), Clear (concise, terse, simple, precise) Correct, Understandable, Feasible (realistic, possible), Independent, Atomic, Necessary, Implementation-free (i.e. abstract). The whole set of requirements should be Consistent, Non-redundant & Complete.

Gathering Requirements

Brainstorming: A group activity designed to generate many diverse and creative ideas for the solution of a problem. There's no bad idea. **Surveys** can be used to solicit responses and opinions from many stakeholders regarding a current product or a new product.

Observing users in their natural work environment can uncover product requirements.

Interviewing stakeholders and domain experts can produce useful information about project requirements.

Focus groups are a kind of informal interview within an interactive group setting. (Beta testers, etc)

Prototyping a mock up, a scaled down version, or a partial system to get users’ feedback, validate a technical concept and give a preview. Can be used for discovering as well as specifying requirements.

Product Survey: Studying existing products can unearth shortcomings of existing solutions that can be addressed by a new product.

Specifying Requirements

Prose: A textual description used to describe requirements. Especially useful when describing abstract ideas such as the vision of a product. AVOID lengthy prose as they are hard to follow.

Feature list: A list of features of a product grouped according to some criteria such as aspect, priority, order of delivery, etc.

User Story (US): User stories are short, simple descriptions of a feature told from the perspective of the person who desires the new capability.

User Story Format: As a {user type/role}, I want to {function} so that I can {benefit}. (Physical/digital medium and {benefit} can be omitted if it's obvious) Epics (Cover bigger functionality) = High-level US.

Usage: User stories capture user requirements in a way that is convenient for scoping (What features to include), estimation (How much effort for the feature) and scheduling (When to deliver the feature).

User stories can capture non-functional requirements too because even NFRs must benefit some stakeholder.

Use case: A use case captures the functional requirements of a system and describes the interaction between the user and the system for a specific functionality of the system. (Use case diagrams)

Actor: An actor (in a use case) is a role played by a user. An actor can be a human or another system. Actors are not part of the system; they reside outside the system.

Glossary: A glossary serves to ensure that all stakeholders have a common understanding of the noteworthy terms, abbreviations, acronyms etc.

Supplementary Requirements Section can be used to capture requirements that do not fit elsewhere. Typically, this is where most Non-Functional Requirements will be listed.

Software Design

Product/External Design: Designing the external behaviour of the product to meet the users' requirements.

Implementation/Internal Design: Designing how the product will be implemented to meet the required external behaviour.

Abstraction requires only details that are relevant to the current perspective or the task at hand. A technique to deal with complexity.

Data abstraction: Abstracting away the lower-level data items and thinking in terms of bigger entities.

Control Abstraction: Abstracting away details of the actual control flow to focus on tasks at a higher level.

Abstraction is not limited to data or control and can be applied repeatedly to obtain progressively higher levels of abstraction.

Coupling is a measure of the degree of dependence between components, classes, methods, etc.

High coupling (aka tight coupling or strong coupling) is discouraged as maintenance, integration, testing and reusing the module is harder and the risk of regression is higher. X is coupled to Y if a change to Y can potentially require a change in X.

Cohesion is a measure of how strongly related and focused the various responsibilities of a component are. **Higher cohesion is better** as the weak/low cohesion lowers the understandability, maintainability and reusability of modules. It can present in many forms but in general, related code is kept together.

Modeling

A **model** is a representation of something else and provides a simpler view of a complex entity because a model captures only a selected aspect. (May need multiple models of the same entity to capture it fully) Usefulness: To analyse a complex entity related to software development. To communicate information among stakeholders. As a blueprint for creating software.

UML is a graphical notation to describe various aspects of a software system.

Modeling Structures

OO Structures: An OO solution is basically a network of objects interacting with each other. Object structure within the same software can change over time by following a class structure (Not random).

UML Object Diagrams model object structures. UML Class Diagrams model class structures. UML class diagrams describe the structure (but not the behaviour) of an OOP solution.

Software Architecture

The software architecture shows the overall organization of the system and can be viewed as a very high-level design. The architecture is typically designed by the software architect and should be simple enough for all team members to understand it.

Architecture diagrams are free-form diagrams. There is no universally adopted standard notation for it. Any reasonable symbols may be used but minimise the variety of symbols and avoid the indiscriminate use of double-headed arrows to show interactions between components.

Architectural Styles: Software architectures follow various high-level styles (aka architectural patterns).

N-tier Architectural Style: In the n-tier style, higher layers make use of services provided by lower layers. Lower layers are independent of higher layers. Follows SoC Principle. (AKA: Multi-layered, Layered)

Client-Server Architectural Style: It has at least one component playing the role of a server and at least one client component accessing the services of the server. It's often used in distributed applications.

Software Design Patterns

Design pattern: An elegant reusable solution to a commonly recurring problem within a given context in software design.

The common format to describe a pattern consists of the following components:

Context: The scenario where the design problem is encountered. **Problem:** The main difficulty to be resolved.

Solution: The core of the solution. It is important to note that the solution presented only includes the most general details, which may need further refinement for a specific context.

Anti-patterns (optional): Commonly used solutions, which are usually incorrect and/or inferior to the Design Pattern.

Consequences (optional): Identifying the pros and cons of applying the pattern.

Other useful information (optional): Code examples, known uses, other related patterns, etc.

Singleton Pattern

Context: Certain classes should have no more than just one instance. These single instances are commonly known as singletons.

Problem: A normal class can be instantiated multiple times by invoking the constructor.

Solution: Make the constructor of the singleton class private, because a public constructor will allow others to instantiate the class at will. Provide a public class-level method to access the single instance.

Pros: Easy to apply. Effective in achieving its goal with minimal extra work. Provides an easy way to access the singleton object from anywhere in the codebase.

Cons: The singleton object acts like a global variable that increases coupling across the codebase. In testing, it is difficult to replace Singleton objects with stubs (static methods cannot be overridden).

In testing, singleton objects carry data from one test to another even when you want each test to be independent of the others. Given that there are some significant cons, it is recommended that you apply the Singleton pattern when, in addition to requiring only one instance of a class, there is a risk of creating multiple objects by mistake, and creating such multiple objects has real negative consequences.

Facade Pattern

Context: Components need to access functionality deep inside other components.

Problem: Access to the component should be allowed without exposing its internal details. E.g. the UI component should access the functionality of the Logic component without knowing that it contains a Book class within it.

Solution: Include a Facade class that sits between the component internals and users of the component such that all access to the component happens through the Facade class.

Design Approaches

Multi-Level Design: In a smaller system, the design of the entire system can be shown in one place. The design of bigger systems needs to be done/shown at multiple levels.

Top-Down & Bottom-Up Design: Within Multi-level design, it can be done in a top-down manner, bottom-up manner, or as a mix.

Top-down: Design the high-level design first and flesh out the lower levels later. Stable high-level before designing low-level.

Bottom-up: Design lower-level components first and put them together to create the higher-level systems later. Not scalable for large systems. May work when designing a variation of an existing system or reusing a lot of existing components to build a new system.

Mix: Design the top levels using the top-down approach but switch to a bottom-up approach when designing the bottom levels.

Integrated Development Environment (IDE)

An IDE generally consists of a source code editor, a compiler/interpreter, a debugger and other tools that aid various aspects of coding.

Debugging is the process of discovering defects in the program.

Code Quality

Production code needs to be of high quality. Aim of a coding standard is to make the entire codebase look like it was written by one person.

Refactoring

Refactoring is the process of improving a program's **internal** structure in **small steps** without modifying its external behaviour. It is not rewriting or bug fixing. It **can** make hidden bugs become easier to spot and improve performance. Refactoring may still result in regressions. Each small refactoring should be followed by **regression testing**. Periodic refactoring is a good way to pay off the technical debt a codebase has accumulated.

Documentation

Developer-to-developer documentation comes in two main forms:

For Developers as Users: This explains how to use software components made by other developers. Some forms are:

- **API Documentation:** Systematic details about individual functions or methods.

- **Tutorials:** High-level guides on how to use the API effectively.

For Developers as Maintainers: This documents how a system or a component is designed, implemented and tested so that other developers can maintain and evolve the code. Usually harder because of the need to explain complex internal details. Given that the reader will have access to the source code, only some information needs to be included.

JavaDoc is a tool for generating API documentation in HTML format from comments in the source code.

Error Handling

Well-written applications include error-handling code that allows them to recover gracefully from unexpected errors.

Exceptions are used to deal with 'unusual' but not entirely unexpected situations that the program might encounter at runtime. When an error occurs at some point in the execution, the code being executed creates an exception object and hands it off to the runtime system. The runtime system then attempts to find something to **handle** (NOT prevent) it in

the call stack. The exception handler chosen is said to catch the exception. **Benefits** includes: The ability to propagate error information through the call stack. The separation of code that deals with 'unusual' situations from the code that does the 'usual' work. In general, **use exceptions only** for 'unusual' conditions. Use normal return statements to pass control to the caller for conditions that are 'normal'.

Assertions are used to define assumptions about the program state so that the runtime can verify them. Drastic actions such as terminating the execution with an error message are typically taken when met with assertion failure as the sooner the execution stops, the safer it is. **It is recommended** that assertions be used generously in the code and do NOT use them to do work as they can be disabled. Assertions are suitable for verifying assumptions about invariants.

Both assertions and exceptions should be used in code. Exception = Unusual condition created by user. Assertion = Mistake by developer.

Logging can be useful for troubleshooting problems and is the deliberate recording of certain information during a program execution for future reference. Able to enable/disable or change logging intensity.

Integration

Integration is combining parts of a software product to form a whole.

Build Automation tools automate the steps of the build process, usually by means of build scripts. Also serve as dependency management tools.

Continuous integration (CI) is the integration, building, and testing that happens automatically after each code change.

Continuous Deployment (CD) is where the changes are not only integrated continuously, but also deployed to end-users at the same time.

Reuse

By reusing tried-and-tested components, the robustness of a new software system can be enhanced while reducing the manpower and time requirement. However, there are costs to it. The reused software may be an overkill, buggy, unstable/immature and risk of dying off.

Application Programming Interface (API) specifies the interface through which other programs can interact with a software component. It is a contract between the component and its clients. By defining API of each component early, the dev team can develop the components in parallel as future behaviour are now more predictable.

Quality Assurance

Software Quality Assurance (QA) is the process of ensuring that the software being built has the required levels of quality. QA = Validation (Build the right system) + Verification (Build the system right).

Whether something belongs under validation or verification is not that important. What is more important is that both are done.

Code review is the systematic examination of code with the intention of finding where the code can be improved. Some examples include:

Pull Request Reviews: Others to review the code in the PR.

In-Pair Programming: Two programmers work on the same code at the same time, implicit reviewing each other code.

Formal Inspections: A group of people systematically examining project artifacts to discover defects.

Static analysis is the analysis of code without executing the code. It can identify unused variables, unhandled exceptions, style errors, etc.

Higher-end static analysis tools (Static Analysers) can perform more complex analysis such as locating potential bugs, memory leaks, inefficient code structures, etc. Linters are a subset of static analysers that aim to locate areas where the code can be made 'cleaner'.

Dynamic Analysis requires the code to be executed to gather additional information about the code. E.g., Performance characteristics, etc.

Formal verification uses mathematical techniques to prove the correctness of a program. It can be used to prove the **absence** of errors. Testing only can prove **presence** of errors. Thus, it is very expensive.

Testing

Testing specifies the input to the software under test (SUT) and the expected behaviour.

Testability is an indication of how easy it is to test an SUT.

Unit Testing: Testing individual units (methods, classes, subsystems) in isolation to ensure each piece works correctly.

Stubs can isolate the SUT from its dependencies so that test are done in isolation. (Other alternatives: Mocks, Fakes, Dummies, Spies)

Integration Testing: Testing whether different parts of the software work together/integrates as expected.

Developers often use a **hybrid** of Unit + Integration tests to minimize the need for stubs.

Alpha Testing is performed by the users, under controlled conditions set by the software development team.

Beta Testing is performed by a selected subset of target users of the system in their natural work setting.

Dogfooding is when creators use their own product to experience how end users experience the product. (Can improve product designs)

Developer Testing is the testing done by the developers themselves as opposed to dedicated testers or end-users. Should be done ASAP as cost of fixing bugs goes up over time.

Scripted testing: Write a set of test cases based on the expected behaviour of the SUT, then perform testing based on that set of test cases.

Exploratory testing: Devise test cases on-the-fly, creating new test cases based on the results of the past test cases. (Also known as reactive testing, error guessing technique, attack-based testing, and bug hunting) Mix of Scripted (Able to discover bugs given enough time) & Exploratory (Depends on tester's experience and intuition) is better.

User Acceptance Testing (UAT): Test the system to ensure it meets the user requirements. Passing UAT ≠ Passing System Testing.

System Testing: Take the whole system and test its **external behaviour** against the system specification. Includes testing against NFRs too. Typically done by a QA team before UAT.

System Testing	Acceptance Testing
Done against system specifications	
Done by Testers	Done by Users/Customers
Done on develop environment	Done on deployment site
+ & - Test Cases	+ Test Cases

The **requirement specification** need not be the same as the **system specification**. However, in many cases one document serves as both.

Regression is the unintended and undesirable effects that may happen on the system due to a modification.

Regression Testing is the re-testing of the software to detect regressions. It is highly recommended to automate regression testing.

IO Redirection Automation: java AddressBook < input.txt > output.txt

Test Driver is the code that 'drives' the SUT for the purpose of testing.

GUI Automation and JUnit Testing (See textbook)

Test Coverage is a metric used to measure the extent to which testing exercises the code.

Function/Method Coverage: Based on functions executed.

Statement Coverage: Based on the number of lines of code executed.

Decision/Branch Coverage: Based on the decision points exercised. E.g., An if statement evaluated to both true and false with separate test cases during testing is considered 'covered'.

Condition Coverage: Based on the Boolean sub-expressions, each evaluated to both true and false with different test cases. Condition coverage is not the same as the decision coverage.

Path Coverage measures coverage in terms of possible paths through a given part of the code executed. 100% path coverage (Highest Intensity) means all possible paths have been executed. Loop ↑ path count greatly.

Entry/Exit Coverage measures coverage in terms of possible calls to

and exits from the operations in the SUT. Measuring coverage is often done using **coverage analysis tools** & can be useful in improving the quality of testing.

Test Case Design (TC Design)

Except for trivial SUTs, exhaustive testing is not practical. For testing to be Efficient & Effective, each new test you add should be targeting a potential fault that is not already targeted by existing test cases.

Black-Box Approach: Test cases are designed exclusively based on the SUT's specified external behaviour.

White-Box (Glass-Box) Approach: Test cases are designed based on what is known about the SUT's implementation, i.e. the code.

Gray-Box Approach: Test case design uses some important information about the implementation.

Equivalence Partition (Equivalence Class): A group of test inputs that are likely to be processed by the SUT in the same way and are usually derived from the specifications of the SUT.

Boundary Value Analysis (BVA) is a test case design heuristic that is based on the observation that bugs often result from incorrect handling of boundaries of equivalence partitions. It suggests that values near boundaries (one from the boundary, one just below the boundary, and one just above the boundary) are more likely to find bugs. Corner Case. An **SUT can take multiple inputs**. Four strategies: All Combinations, At Least Once, All Pairs & Random.

Each Valid Input must be included at least once in a Positive TC. **Invalid Test Inputs** should be tested individually first before combining. It is **impossible** to determine the precise number of test cases required. **Use Cases** can be used for system testing and acceptance testing.

Project Management

Stages such as requirements, analysis, design, implementation and testing are known as the **Software Development Lifecycle (SDLC)**. The **sequential model**, also called the **waterfall model**, views software development as a linear (depth-first) process.

The **iterative model** advocates producing the software by going through several iterations (breadth-first or depth-first).

Agile Models value individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation & responding to change over following a plan. It may not be suitable for some projects.

Extreme Programming (XP) stresses customer satisfaction & teamwork. It aims to empower developers to confidently respond to changing customer requirements and improve a software project using communication, simplicity, feedback, respect, and courage. It has a set of simple rules.

Scrum is a process skeleton that contains sets of practices and predefined roles. Each sprint starts with a planning meeting and aims to create a potentially deliverable product incrementally. Daily scrum meeting (NOT used for problem solving) are held in the morning (15 mins + 3 questions).

Software Principles

Single Responsibility Principle (SRP): A class should have one, and only one, reason to change.

Open-Closed Principle (OCP): A module should be open for extension but closed for modification. That is, modules should be written so that they can be extended, without requiring them to be modified.

Liskov Substitution Principle (LSP): Derived classes must be substitutable for their base classes.

Separation of Concerns Principle (SoC): To achieve better modularity, separate the code into distinct sections, such that each section addresses a separate concern. It reduces functional overlaps among code sections and limits the ripple effect. It can be applied to class level as well as at higher levels. It leads to higher cohesion and lower coupling.